

Historias de Usuario — Food Store E-Commerce

- > ****Proyecto****: Plataforma de e-commerce de alimentos
- > ****Fecha****: 2026-03-31
- > ****Ordenamiento****: Por orden lógico de implementación (dependencias resueltas primero)

Actores del Sistema

Actor	Descripcion	
-----	-----	
Cliente	Usuario final que navega, compra y gestiona sus pedidos	
Admin	Acceso total al sistema: usuarios, catalogo, pedidos, metricas, config	
Gestor de Stock	Gestiona productos, ingredientes, alergenosen, stock y categorias	
Gestor de Pedidos	Visualiza y gestiona el ciclo de vida de los pedidos (FSM)	
Sistema	Procesos automaticos: webhooks, transiciones de estado, tokens	

Reglas de Negocio (Referencia Completa)

****Dominio: Autenticación y Seguridad****

ID	Regla	Historias Asociadas
RN-AU01	La contraseña NUNCA se almacena en texto plano; se hasha con bcrypt (cost factor >= 10) con salt automático	US-001, US-063
RN-AU02	El access token JWT tiene duración de 30 minutos, contiene userId, email y roles, firmado con HS256	US-002, US-003
RN-AU03	El refresh token tiene duración de 7 días, es un UUID v4 opaco almacenado en BD	US-002, US-003

| RN-AU04 | Al usar un refresh token se aplica rotación: el anterior se revoca y se emite uno nuevo | US-003 |

| RN-AU05 | Si se detecta reuso de un refresh token ya utilizado (replay attack), se revocan TODOS los tokens del usuario | US-003 |

| RN-AU06 | Rate limiting en login: máximo 5 intentos por IP en ventana de 15 minutos; excedido retorna HTTP 429 | US-002, US-073 |

| RN-AU07 | Al registrarse se asigna automáticamente el rol CLIENT; el rol NO viene del request | US-001 |

| RN-AU08 | La respuesta de login NO debe diferenciar "email no existe" de "contraseña incorrecta" (seguridad) | US-002 |

| RN-AU09 | Los datos sensibles de tarjetas NUNCA pasan por el servidor de Food Store (PCI DSS SAQ-A) | US-045 |

| RN-AU10 | El archivo .env con secrets NUNCA se commitea al repositorio | US-000 |

****Dominio: Autorización y Roles (RBAC)****

ID	Regla	Historias Asociadas
-----	-----	-----
RN-RB01	Existen 4 roles fijos con IDs estables: ADMIN (1), STOCK (2), PEDIDOS (3), CLIENT (4)	US-000b, US-005
RN-RB02	Un usuario puede tener múltiples roles simultáneamente (M2M con restricción UNIQUE compuesta)	US-005
RN-RB03	Solo ADMIN puede asignar/modificar roles de otros usuarios	US-005, US-054
RN-RB04	Un ADMIN no puede quitarse el rol ADMIN a sí mismo si es el último administrador del sistema	US-005, US-054
RN-RB05	Un CLIENT solo puede ver y operar sobre sus propios datos, nunca los de otros usuarios	US-049, US-050, US-025, US-061
RN-RB06	Gestor de Stock NO tiene acceso a pedidos, usuarios ni métricas	US-006, US-075
RN-RB07	Gestor de Pedidos NO tiene acceso a catálogo ni gestión de usuarios	US-006, US-075
RN-RB08	Solo ADMIN puede cancelar pedidos en estado EN_PREPARACIÓN	US-043
RN-RB09	Si el usuario no posee el rol requerido, el sistema retorna HTTP 403 Forbidden	US-006, US-076

| RN-RB10 | Endpoint sin token válido retorna HTTP 401; rutas públicas (catálogo, login, registro) no requieren auth | US-006, US-076 |

****Dominio: Catálogo de Productos****

| ID | Regla | Historias Asociadas

| RN-CA01 | Las categorías soportan jerarquía de profundidad arbitraria mediante FK autoreferencial (padre_id) | US-007, US-008 |

| RN-CA02 | No se permite asignar una categoría como padre de sí misma ni generar ciclos en la jerarquía | US-009 |

| RN-CA03 | No se puede eliminar una categoría que tenga productos activos asociados | US-010 |

| RN-CA04 | El precio del producto se almacena como NUMERIC de precisión fija (nunca float/double) | US-015, US-020 |

| RN-CA05 | El stock es un entero ≥ 0 ; nunca puede ser negativo | US-015, US-021 |

| RN-CA06 | Un producto puede pertenecer a múltiples categorías (M2M vía ProductoCategoria) | US-016 |

| RN-CA07 | Un producto puede tener múltiples ingredientes (M2M vía ProductoIngrediente); cada ingrediente tiene flag es_alergeno | US-017 |

| RN-CA08 | El catálogo público solo muestra productos con disponible=true y eliminado_en IS NULL | US-018 |

| RN-CA09 | El soft delete marca eliminado_en con timestamp; NUNCA se borra físicamente (preserva integridad referencial) | US-010, US-014, US-022, US-055 |

| RN-CA10 | Los endpoints de admin pueden incluir parámetro incluir_eliminados para ver registros borrados lógicamente | US-064 |

****Dominio: Direcciones de Entrega****

ID	Regla	Historias Asociadas
RN-DI01	Un cliente puede tener múltiples direcciones; la primera se marca como predeterminada automáticamente	US-024
RN-DI02	Solo una dirección puede ser predeterminada a la vez por usuario	US-028
RN-DI03	Un cliente solo puede ver/editar/eliminar sus propias direcciones (ownership por userId del JWT)	US-025, US-026, US-027

****Dominio: Carrito de Compras****

ID	Regla	Historias Asociadas
RN-CR01	El carrito es client-side only (Zustand + localStorage); no existe en el backend	US-029 a US-034
RN-CR02	El carrito persiste al cerrar el navegador, refresh de página, y logout/login	US-029
RN-CR03	Si un producto ya está en el carrito y se agrega de nuevo, se incrementa la cantidad (no se duplica)	US-029
RN-CR04	Solo se pueden excluir ingredientes que el producto efectivamente tiene asociados	US-030
RN-CR05	La personalización (exclusión de ingredientes) se almacena como array de IDs de ingredientes	US-030, US-035

****Dominio: Pedidos — Creación****

ID	Regla	Historias Asociadas
RN-PE01	La creación de un pedido es ATÓMICA (Unit of Work): si falla cualquier parte, no se persiste nada	US-035, US-036
RN-PE02	Al crear un pedido se genera snapshot del precio de cada producto (precio_snapshot en DetallePedido)	US-035, US-037
RN-PE03	Al crear un pedido se genera snapshot de la dirección de entrega (direccion_snapshot en Pedido)	US-035, US-038
RN-PE04	Se debe validar stock suficiente DENTRO de la transacción (SELECT FOR UPDATE) antes de crear el pedido	US-036

| RN-PE05 | Si algún producto no tiene stock suficiente, no se crea NINGÚN ítem del pedido (todo o nada) | US-036 |

| RN-PE06 | Todo pedido nace en estado PENDIENTE con registro inicial en HistorialEstadoPedido | US-035 |

| RN-PE07 | La personalización se almacena como INTEGER[] (array de PostgreSQL) en DetallePedido | US-035 |

| RN-PE08 | El total del pedido = suma de subtotales (cantidad x precio_snapshot) + costo de envío | US-035 |

****Dominio: Pedidos — Máquina de Estados (FSM)****

ID	Regla	Historias Asociadas
----	-------	---------------------

RN-FS01 Un pedido solo puede avanzar al siguiente estado en la secuencia; no se permiten saltos ni retrocesos	US-039 a US-042	
---	-----------------	--

RN-FS02 La transición PENDIENTE → CONFIRMADO es EXCLUSIVAMENTE automática (por pago aprobado); nadie la ejecuta manual	US-039, US-046	
--	----------------	--

RN-FS03 Al confirmar (PENDIENTE→CONFIRMADO), se decrementa atómicamente el stock de cada producto del pedido	US-039	
--	--------	--

RN-FS04 Si el decremento de stock falla para cualquier producto, toda la operación se revierte (rollback)	US-039	
---	--------	--

RN-FS05 Al cancelar un pedido que ya fue CONFIRMADO, se debe restaurar el stock de forma atómica (operación inversa)	US-043	
--	--------	--

RN-FS06 ENTREGADO y CANCELADO son estados terminales; no se permite ninguna transición adicional desde ellos	US-042, US-043	
--	----------------	--

RN-FS07 Todo cambio de estado se registra en HistorialEstadoPedido (append-only: solo INSERT, nunca UPDATE ni DELETE)	US-039 a US-044	
---	-----------------	--

RN-FS08 Cancelación posible desde: PENDIENTE (Cliente/Gestor/Admin), CONFIRMADO (Gestor/Admin), EN_PREPARACIÓN (solo Admin)	US-043	
---	--------	--

RN-FS09 Cada registro de historial incluye: estado anterior, estado nuevo, timestamp, usuario o SISTEMA, observación	US-044	
--	--------	--

****Dominio: Pagos — MercadoPago****

ID	Regla	Historias Asociadas
----	-------	---------------------

| RN-PA01 | Los datos de tarjeta se tokenizan en el browser via SDK MercadoPago.js (nunca tocan nuestro servidor) | US-045 |

| RN-PA02 | Cada pago tiene un idempotency_key único; si se recibe webhook duplicado con misma key, se ignora | US-045, US-046 |

| RN-PA03 | El webhook debe responder HTTP 200 inmediatamente para evitar reintentos de MercadoPago | US-046 |

| RN-PA04 | Siempre se verifica el estado real consultando la API de MercadoPago; nunca se confía solo en los datos del webhook | US-046 |

| RN-PA05 | Pago "approved" → transición automática PENDIENTE→CONFIRMADO + decremento de stock | US-046 |

| RN-PA06 | Pago "rejected" → pedido permanece PENDIENTE; el cliente puede reintentar con otro método | US-046, US-048 |

| RN-PA07 | Pago "pending"/"in_process" → se actualiza estado del pago pero el pedido sigue PENDIENTE | US-046 |

| RN-PA08 | Un pedido puede tener múltiples intentos de pago (relación 1:N Pedido→Pago) | US-048 |

| RN-PA09 | Se usa external_reference para vincular la preferencia de MercadoPago con el pedido en Food Store | US-045, US-046 |

****Dominio: Datos e Integridad****

ID	Regla	Historias Asociadas
RN-DA01	Todas las tablas principales tienen campos de auditoría: creado_en (default NOW) y actualizado_en (auto-update)	US-000b
RN-DA02	Los IDs de seed (Roles, EstadoPedido) son ESTABLES y explícitos; se referencian en el código	US-000b
RN-DA03	El script de seed es idempotente: ejecutarlo múltiples veces no duplica datos	US-000b
RN-DA04	El email del usuario tiene restricción UNIQUE e índice para optimizar búsquedas en login	US-001, US-002
RN-DA05	El HistorialEstadoPedido es append-only: NUNCA se actualiza ni se elimina un registro	US-044
RN-DA06	Los snapshots garantizan inmutabilidad: cambios futuros en productos/direcciones NO afectan pedidos existentes	US-037, US-038
RN-DA07	La paginación usa skip/limit con total de registros para que el frontend construya controles	US-018, US-049, US-051

| RN-DA08 | Los errores de API siguen el estándar RFC 7807 (Problem Details for HTTP APIs)
| US-068 |

HISTORIAS DE USUARIO

EPIC 00 — Infraestructura y Setup (Sprint 0)

> **Sprint 0** — Sin esta epica no existe NADA. Es la fundacion sobre la que se construye todo lo demas. Cada historia posterior depende directa o indirectamente de estas.

US-000: Inicializacion del repositorio y estructura del proyecto

- **Titulo**: Scaffolding del monorepo y estructura base
- **Historia**: Como **Lider Tecnico**, quiero tener el repositorio Git inicializado con la estructura de carpetas del backend (feature-first) y del frontend (Feature-Sliced Design), para que el equipo pueda comenzar a desarrollar sobre una base organizada y consistente.
- **Prioridad**: Alta
- **Dependencias**: Ninguna

Criterios de Aceptacion:

- [] GIVEN un repositorio vacio, WHEN se ejecuta el setup inicial, THEN existe un monorepo con carpetas `/backend` y `/frontend` claramente separadas.
- [] El backend tiene la estructura feature-first: carpetas por modulo (`auth/`, `usuarios/`, `productos/`, `categorias/`, `ingredientes/`, `pedidos/`, `pagos/`, `direcciones/`, `admin/`, `refreshtokens/`), cada una con sus archivos `model.py`, `schemas.py`, `repository.py`, `service.py`, `router.py`.
- [] El frontend tiene la estructura FSD: `app/`, `pages/`, `widgets/`, `features/`, `entities/`, `shared/`.
- [] Existe un `.gitignore` que excluye: `.env`, `\_\_pycache\_\_`, `node\_modules/`, `.venv/`, `\*.pyc`, `dist/`, `DS\_Store`.
- [] Existe un `README.md` raiz con instrucciones basicas de setup (clonar, instalar, ejecutar).
- [] Existe un `.env.example` en backend y frontend con todas las variables documentadas y valores de ejemplo.
- [] El historial de Git muestra commits progresivos (no un solo commit masivo).

Notas Tecnicas:

- Backend: Python con FastAPI, estructura modular vertical
- Frontend: React + TypeScript + Vite
- Convencion de commits: conventional commits

US-000a: Configuracion del entorno backend (FastAPI + dependencias)

- **Titulo**: Setup del backend con FastAPI y dependencias core
- **Historia**: Como **Desarrollador**, quiero tener el proyecto backend configurado con FastAPI, SQLAlchemy, Alembic y todas las dependencias necesarias, para poder comenzar a implementar los modulos funcionales.
- **Prioridad**: Alta
- **Dependencias**: US-000

Criterios de Aceptacion:

- [] GIVEN el proyecto backend, WHEN se ejecuta `pip install -r requirements.txt` (o `poetry install`), THEN se instalan todas las dependencias: FastAPI, SQLAlchemy, Alembic, Passlib[bcrypt], python-jose, slowapi, mercadopago, uvicorn, httpx, pydantic[email-validator].
- [] GIVEN el proyecto backend, WHEN se ejecuta `uvicorn main:app --reload`, THEN el servidor arranca sin errores en el puerto 8000.
- [] La documentacion Swagger es accesible en `/docs` y ReDoc en `/redoc`.
- [] Existe un archivo `main.py` que configura la app FastAPI con: CORS middleware (origenes desde variable de entorno), rate limiting middleware, y registro de routers con prefijo `/api/v1`.
- [] Existe un modulo `core/` o `config/` con: `config.py` (lectura de variables de entorno con valores por defecto), `database.py` (engine y session factory de SQLAlchemy), `security.py` (funciones de hashing y JWT).
- [] El CORS permite el origen `http://localhost:5173` en desarrollo.

Notas Tecnicas:

- Variables de entorno: DATABASE_URL, SECRET_KEY, JWT_ACCESS_TOKEN_EXPIRE_MINUTES (30), JWT_REFRESH_TOKEN_EXPIRE_DAYS (7), CORS_ORIGINS, MERCADOPAGO_ACCESS_TOKEN, MERCADOPAGO_PUBLIC_KEY
- Base de datos: PostgreSQL, connection string en DATABASE_URL
- Middleware de errores: RFC 7807 (Problem Details for HTTP APIs)

US-000b: Configuración de PostgreSQL, migraciones y seed data

- **Titulo**: Base de datos, migraciones Alembic y datos semilla
- **Historia**: Como **Desarrollador**, quiero tener PostgreSQL configurado con Alembic para migraciones y un script de seed que cargue los datos iniciales, para que el sistema tenga las tablas y datos catalogo necesarios para funcionar.
- **Prioridad**: Alta
- **Dependencias**: US-000a

Criterios de Aceptación:

- [] GIVEN una base de datos PostgreSQL vacía, WHEN se ejecuta `alembic upgrade head`, THEN se crean TODAS las tablas del ERD v5 sin errores: Usuario, Rol, UsuarioRol, RefreshToken, DireccionEntrega, Categoria, Producto, Ingrediente, ProductoCategoria, ProductoIngrediente, FormaPago, EstadoPedido, Pedido, DetallePedido, HistorialEstadoPedido, Pago.
- [] Todas las tablas principales tienen campos de auditoria: `creado\_en` (timestamp, default NOW), `actualizado\_en` (timestamp, auto-update).
- [] Las tablas que soportan soft delete tienen el campo `eliminado\_en` (timestamp nullable).
- [] Los tipos de datos son correctos: precios como NUMERIC de precision fija, personalizacion como INTEGER[], email con constraint UNIQUE, etc.
- [] Las claves foraneas y restricciones de integridad referencial estan definidas.
- [] Categoria tiene `padre\_id` como FK autoreferencial nullable.
- [] UsuarioRol tiene restriccion UNIQUE compuesta (usuario_id, rol_id).
- [] GIVEN las tablas creadas, WHEN se ejecuta el script de seed, THEN se cargan:
 - 4 Roles: ADMIN (1), STOCK (2), PEDIDOS (3), CLIENT (4)
 - 6 EstadoPedido: PENDIENTE (1), CONFIRMADO (2), EN_PREPARACION (3), EN_CAMINO (4), ENTREGADO (5), CANCELADO (6)
 - Formas de pago: Tarjeta de credito, Tarjeta de debito (activas)
 - 1 Usuario administrador con rol ADMIN y credenciales configurables por variables de entorno
- [] Las migraciones son reversibles: `alembic downgrade -1` no genera errores.
- [] El script de seed es idempotente: ejecutarlo multiples veces no duplica datos.

****Reglas de Negocio****: Los IDs de Roles y EstadoPedido son estables y se referencian en el codigo.

****Notas Tecnicas****:

- Alembic con autogenerate desde modelos SQLAlchemy
- Seed como script Python invocable: `python -m scripts.seed` o similar
- Usar `INSERT ... ON CONFLICT DO NOTHING` para idempotencia
- Los IDs de seed deben ser explicitos (no autogenerados) para consistencia

US-000c: Configuración del entorno frontend (React + Vite + dependencias)

- ****Titulo****: Setup del frontend con React, TypeScript, Vite y dependencias core
- ****Historia****: Como ****Desarrollador****, quiero tener el proyecto frontend configurado con React, TypeScript, Vite y todas las librerías necesarias, para poder comenzar a construir la interfaz de usuario.
- ****Prioridad****: Alta
- ****Dependencias****: US-000

****Criterios de Aceptación****:

- [] GIVEN el proyecto frontend, WHEN se ejecuta `npm install`, THEN se instalan todas las dependencias: react, react-dom, react-router-dom, @tanstack/react-query, @tanstack/react-form, zustand, axios, recharts, tailwindcss, @mercadopago/sdk-js.
- [] GIVEN el proyecto frontend, WHEN se ejecuta `npm run dev`, THEN el servidor de desarrollo arranca sin errores en el puerto 5173.
- [] TypeScript está configurado en modo estricto (`strict: true` en `tsconfig.json`).
- [] Tailwind CSS está configurado con PostCSS y purging de clases en producción.
- [] Existe la configuración de Axios con:
 - Base URL desde variable de entorno `VITE\_API\_BASE\_URL`
 - Interceptor de request que adjunta el access token del authStore al header `Authorization: Bearer <token>`

- Interceptor de response que, ante un 401, intenta refresh automatico con el refresh token, actualiza el authStore, y reintenta la peticion original
- [] Existe el archivo `.env.example` con: `VITE\_API\_BASE\_URL=http://localhost:8000/api/v1`, `VITE\_MERCADOPAGO\_PUBLIC\_KEY=TEST-xxx`.
- [] El routing base esta configurado con react-router-dom (rutas publicas y privadas).
- [] TanStack Query esta configurado con un `QueryClientProvider` en el App root.

****Notas Tecnicas**:**

- Vite con plugin React + SWC para fast refresh
- Tailwind v3+ con PostCSS
- Axios instance centralizada en `shared/api/axios.ts`
- QueryClient con defaults razonables: staleTime, retry, refetchOnWindowFocus

US-000d: Implementacion de patrones base (BaseRepository, Unit of Work, dependencias FastAPI)

- ****Titulo****: Patrones de infraestructura del backend
- ****Historia****: Como ****Desarrollador****, quiero tener implementados el BaseRepository generico, el Unit of Work y las dependencias de FastAPI (get_current_user, require_role), para que los modulos funcionales puedan construirse sobre una base solida y consistente.
- ****Prioridad****: Alta
- ****Dependencias****: US-000b

****Criterios de Aceptacion**:**

- [] Existe un `BaseRepository[T]` generico con los metodos: `get\_by\_id(id)`, `list\_all(skip, limit, filters)`, `count(filters)`, `create(obj)`, `update(id, data)`, `soft\_delete(id)`, `hard\_delete(id)`.
- [] `get\_by\_id` y `list\_all` excluyen registros con `eliminado\_en IS NOT NULL` por defecto.
- [] Existe un `UnitOfWork` implementado como context manager (`async with`) que:
 - Crea una sesion de SQLAlchemy al entrar
 - Expone los repositorios como atributos (`uow.productos`, `uow.pedidos`, etc.)
 - Ejecuta `commit()` al salir exitosamente

- Ejecuta ``rollback()`` automaticamente si se lanza una excepcion
- [] Existe la dependencia ``get_current_user`` que:
 - Extrae el token del header ``Authorization: Bearer <token>``
 - Decodifica y valida el JWT (firma, expiracion)
 - Retorna el objeto Usuario o lanza HTTP 401
- [] Existe la dependencia factory ``require_role(roles: list[str])`` que:
 - Recibe una lista de roles permitidos
 - Verifica que el usuario autenticado tenga al menos uno de esos roles
 - Lanza HTTP 403 si no tiene permiso
- [] Existe un middleware o handler de excepciones que formatea errores segun RFC 7807 con campos: ``type``, ``title``, ``status``, ``detail``, ``instance``.

****Notas Tecnicas**:**

- BaseRepository parametrizado con TypeVar para tipado generico
- UoW inicializa repos en ``__aenter__``, commit/rollback en ``__aexit__``
- ``get_current_user`` usa ``Depends(oauth2_scheme)`` de FastAPI
- ``require_role`` retorna un ``Callable`` que FastAPI puede usar como dependencia

US-000e: Configuracion de los stores de Zustand (authStore, cartStore, paymentStore, uiStore)

- ****Titulo****: Stores de estado del cliente con Zustand
- ****Historia****: Como ****Desarrollador****, quiero tener los cuatro stores de Zustand configurados con sus acciones base y persistencia, para que el frontend tenga una gestion de estado consistente desde el inicio.
- ****Prioridad****: Alta
- ****Dependencias****: US-000c

****Criterios de Aceptacion**:**

- [] Existe ``authStore`` con:
 - Estado: ``accessToken``, ``refreshToken``, ``user`` (id, nombre, email, roles), ``isAuthenticated``

- Acciones: ``login(tokens, user)``, ``logout()``, ``updateTokens(tokens)``
- Selectores: ``isAuthenticated()``, ``hasRole(role)``
- Persistencia en `localStorage` con clave ``food-store-auth``
- ``partialize`` que excluye estados transitorios (`isLoading`)
- [] Existe ``cartStore`` con:
 - Estado: ``items`` (array de `{productold, producto, cantidad, personalizacion}`)
 - Acciones: ``addItem(producto, cantidad, personalizacion)``, ``removeItem(productold)``, ``updateQuantity(productold, cantidad)``, ``clearCart()``
 - Selectores: ``totalItems()``, ``totalPrice()``, ``getItem(productold)``
 - Persistencia en `localStorage` con clave ``food-store-cart``
 - El carrito sobrevive al cierre del navegador, refresh de pagina, y logout/login
- [] Existe ``paymentStore`` con:
 - Estado: ``checkoutStep``, ``preferenceld``, ``paymentStatus``, ``error``
 - Acciones: ``startCheckout(pedidold)``, ``setPreference(preferenceld)``, ``updatePaymentStatus(status)``, ``resetPayment()``
 - SIN persistencia en `localStorage` (estado transitorio)
- [] Existe ``uiStore`` con:
 - Estado: ``theme`` (light/dark), ``sidebarOpen``, ``toasts``
 - Persistencia selectiva: solo ``theme`` se persiste
- [] Todos los stores usan suscripcion por slice (no ``useStore()`` completo).

****Notas Tecnicas**:**

- Cada store en su propio archivo dentro de ``shared/stores/`` o ``entities/``
- Usar ``useStore.getState()`` en el interceptor de Axios (fuera de React)
- Middleware persist con ``partialize`` para filtrar que se guarda

US-068: Manejo de errores estandarizado en backend

- ****Titulo****: Formato de error consistente en API

- **Historia**: Como **Sistema**, quiero que todos los errores de la API sigan un formato consistente, para facilitar el manejo en el frontend y debugging.
- **Prioridad**: Alta
- **Dependencias**: Ninguna

Criterios de Aceptacion:

- [] Todos los errores siguen el formato: `{ statusCode, message, errors?, timestamp }`.
- [] Los errores de validacion incluyen detalle por campo: `{ field, message }[]`.
- [] Los errores no exponen stack traces ni detalles de implementacion en produccion.
- [] Se loguean errores 500 con stack trace en el servidor.

Notas Tecnicas:

- Exception filter/middleware global
- Clases de error custom: `ValidationError`, `UnauthorizedError`, `ForbiddenError`, `NotFoundError`
- Patron: mapear excepciones a HTTP status codes en un solo lugar

US-074: Validacion y sanitizacion de inputs

- **Titulo**: Proteccion contra inyeccion y datos malformados
- **Historia**: Como **Sistema**, quiero validar y sanitizar todos los inputs del usuario, para prevenir inyecciones SQL/XSS y datos corruptos.
- **Prioridad**: Alta
- **Dependencias**: Ninguna

Criterios de Aceptacion:

- [] Todos los inputs se validan en backend con esquemas (Zod, class-validator o similar).
- [] Los strings se sanitizan contra XSS (escape de HTML entities).
- [] Los queries parametrizados previenen SQL injection (ORM con bindings).
- [] Los campos numericos rechazan valores no numericos con error 400.

****Notas Tecnicas**:**

- Validation pipe global (NestJS) o middleware equivalente
- ORM con queries parametrizados (nunca concatenar SQL)
- Library de sanitizacion: DOMPurify para frontend, helmet para headers

EPIC 01 — Autenticacion y Autorizacion

US-001: Registro de cliente

- ****Titulo****: Registro de nuevo cliente
- ****Historia****: Como ****Cliente****, quiero registrarme en la plataforma con mi email y contrasena, para poder acceder a las funcionalidades de compra.
- ****Prioridad****: Alta
- ****Dependencias****: US-000a, US-000b, US-000d (backend configurado, BD con tablas y seed data, patrones base implementados)

****Criterios de Aceptacion**:**

- [] GIVEN un usuario no registrado, WHEN completa el formulario con nombre, email y contrasena validos, THEN se crea la cuenta con rol CLIENT asignado automaticamente.
- [] GIVEN un email ya registrado, WHEN intenta registrarse con ese email, THEN el sistema muestra error "El email ya esta registrado".
- [] GIVEN una contrasena con menos de 8 caracteres, WHEN intenta registrarse, THEN el sistema rechaza el registro con mensaje de validacion.
- [] La contrasena se almacena hasheada con bcrypt (cost factor >= 10).
- [] Al registrarse exitosamente, se retorna un par de tokens (access + refresh).

****Reglas de Negocio****: RN-AU01, RN-AU07, RN-DA04.

****Notas Tecnicas**:**

- Endpoint: `POST /api/auth/register`

- Hashing: bcrypt con salt automatico
- Rol CLIENT se asigna en la capa de servicio, no viene del request
- Validacion de email con regex RFC 5322 simplificado

US-002: Login de usuario

- **Titulo**: Inicio de sesion
- **Historia**: Como **Cliente**, quiero iniciar sesion con mis credenciales, para acceder a mi cuenta y realizar compras.
- **Prioridad**: Alta
- **Dependencias**: US-001

Criterios de Aceptacion:

- [] GIVEN credenciales validas, WHEN el usuario envia email y contrasena, THEN recibe un access token (30 min) y un refresh token (7 dias).
- [] GIVEN credenciales invalidas, WHEN intenta loguearse, THEN recibe error 401 sin revelar si el email existe o no.
- [] GIVEN 5 intentos fallidos en 15 minutos desde la misma IP, WHEN intenta un 6to login, THEN recibe error 429 "Demasiados intentos, reintenta en X minutos".
- [] El access token contiene: userId, email, rol, exp.
- [] El refresh token se almacena de forma segura (httpOnly cookie o storage seguro).

Reglas de Negocio: RN-AU02, RN-AU06, RN-AU08, RN-DA04.

Notas Tecnicas:

- Endpoint: `POST /api/auth/login`
- JWT firmado con RS256 o HS256 segun config
- Rate limiting implementado con sliding window (Redis o in-memory)
- Respuesta no debe diferenciar "email no existe" de "contrasena incorrecta"

US-003: Refresh de token

- **Titulo**: Renovacion automatica de sesion
- **Historia**: Como **Sistema**, quiero rotar los tokens de acceso usando el refresh token, para mantener la sesion del usuario activa de forma segura.
- **Prioridad**: Alta
- **Dependencias**: US-002

Criterios de Aceptacion:

- [] GIVEN un refresh token valido y no expirado, WHEN se envia al endpoint de refresh, THEN se genera un nuevo par access + refresh token y se invalida el refresh token anterior (rotacion).
- [] GIVEN un refresh token expirado, WHEN se envia al endpoint, THEN se retorna 401 y el usuario debe re-loguearse.
- [] GIVEN un refresh token ya utilizado (replay attack), WHEN se envia, THEN se invalidan TODOS los refresh tokens del usuario y se retorna 401.
- [] El nuevo refresh token tiene una nueva fecha de expiracion (7 dias desde la emision).

Notas Tecnicas:

- Endpoint: `POST /api/auth/refresh`
- Familia de tokens: cada refresh token tiene un `familyId` para detectar reuso
- Almacenamiento de refresh tokens en BD con flag `used`

US-004: Logout

- **Titulo**: Cierre de sesion
- **Historia**: Como **Cliente**, quiero cerrar mi sesion, para proteger mi cuenta cuando dejo de usar la plataforma.
- **Prioridad**: Media
- **Dependencias**: US-002

****Criterios de Aceptacion**:**

- [] GIVEN un usuario autenticado, WHEN solicita logout, THEN se invalida el refresh token actual.
- [] GIVEN un access token post-logout, WHEN se usa para una request, THEN sigue siendo valido hasta su expiracion natural (stateless).
- [] El cliente limpia los tokens del storage local.

****Notas Tecnicas**:**

- Endpoint: `POST /api/auth/logout`
- Invalidar refresh token en BD (soft delete o flag)
- Frontend: limpiar Zustand auth store + localStorage

US-005: Gestion de roles (RBAC)

- ****Titulo****: Asignacion y verificacion de roles
- ****Historia****: Como ****Admin****, quiero asignar roles a los usuarios del sistema, para controlar el acceso a las distintas funcionalidades.
- ****Prioridad****: Alta
- ****Dependencias****: US-001

****Criterios de Aceptacion**:**

- [] GIVEN un Admin autenticado, WHEN asigna un rol (ADMIN, STOCK, PEDIDOS, CLIENT) a un usuario, THEN el usuario obtiene los permisos correspondientes.
- [] GIVEN un usuario con rol CLIENT, WHEN intenta acceder a un endpoint de Admin, THEN recibe 403 Forbidden.
- [] Los 4 roles son: ADMIN, STOCK, PEDIDOS, CLIENT.
- [] Solo ADMIN puede modificar roles de otros usuarios.
- [] Un ADMIN no puede quitarse el rol ADMIN a si mismo si es el ultimo admin.

****Notas Tecnicas**:**

- Endpoint: `PUT /api/admin/users/:id/role`
- Middleware de autorizacion: `@Roles('ADMIN')` o guard equivalente
- Decorador/middleware que extrae el rol del JWT y valida contra la ruta

US-006: Proteccion de rutas por rol

- **Titulo**: Middleware de autorizacion por rol
- **Historia**: Como **Sistema**, quiero proteger cada endpoint segun el rol requerido, para garantizar que solo usuarios autorizados accedan a cada recurso.
- **Prioridad**: Alta
- **Dependencias**: US-005

Criterios de Aceptacion:

- [] GIVEN un request sin token, WHEN accede a una ruta protegida, THEN recibe 401.
- [] GIVEN un token valido con rol insuficiente, WHEN accede a una ruta restringida, THEN recibe 403.
- [] GIVEN un token expirado, WHEN accede a una ruta protegida, THEN recibe 401 con mensaje indicando expiracion.
- [] Las rutas publicas (catalogo, login, registro) no requieren autenticacion.

Reglas de Negocio: Mapeo de roles a rutas segun tabla de permisos.

Notas Tecnicas:

- Middleware/Guard global con lista blanca de rutas publicas
- Decorador de metadata para roles requeridos por endpoint
- Patron: extraer claims del JWT -> verificar rol contra metadata de la ruta

US-073: Rate limiting en endpoints sensibles

- **Titulo**: Proteccion contra abuso en endpoints criticos
- **Historia**: Como **Sistema**, quiero limitar la tasa de requests en endpoints sensibles, para proteger el sistema contra ataques de fuerza bruta y abuso.
- **Prioridad**: Alta
- **Dependencias**: US-002

Criterios de Aceptacion:

- [] Login: maximo 5 intentos por IP en ventana de 15 minutos.
- [] Registro: maximo 3 registros por IP en ventana de 1 hora.
- [] Creacion de pedido: maximo 10 por usuario por hora.
- [] Al exceder el limite, respuesta 429 con header `Retry-After`.

Notas Tecnicas:

- Middleware de rate limiting con sliding window
- Almacenamiento: Redis (produccion) o Map in-memory (desarrollo)
- Headers: `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`

EPIC 02 — Navegacion y Layout Base

US-075: Navegacion por rol

- **Titulo**: Menu adaptado al rol del usuario
- **Historia**: Como **usuario del sistema**, quiero ver solo las opciones de menu correspondientes a mi rol, para tener una interfaz limpia y enfocada en mis tareas.
- **Prioridad**: Alta
- **Dependencias**: US-006

Criterios de Aceptacion:

- [] GIVEN un usuario con rol CLIENT, WHEN ve el menu, THEN ve: Catalogo, Mi Carrito, Mis Pedidos, Mi Perfil, Mis Direcciones.
- [] GIVEN un usuario con rol STOCK, WHEN ve el menu, THEN ve: Productos, Categorias, Ingredientes, Stock.
- [] GIVEN un usuario con rol PEDIDOS, WHEN ve el menu, THEN ve: Panel de Pedidos.
- [] GIVEN un usuario con rol ADMIN, WHEN ve el menu, THEN ve: todas las opciones de todos los roles + Usuarios + Metricas + Configuracion.
- [] Un usuario no autenticado ve: Catalogo, Login, Registrarse.

****Notas Tecnicas**:**

- Componente: `Navigation` / `Sidebar`
- Guard de rutas en frontend basado en rol del JWT decodificado
- Lazy loading de modulos por rol

US-076: Proteccion de rutas en frontend

- ****Titulo****: Guards de navegacion por autenticacion y rol
- ****Historia****: Como ****Sistema****, quiero proteger las rutas del frontend segun autenticacion y rol, para evitar que usuarios accedan a vistas no autorizadas.
- ****Prioridad****: Alta
- ****Dependencias****: US-075

****Criterios de Aceptacion**:**

- [] GIVEN un usuario no autenticado, WHEN intenta acceder a una ruta protegida, THEN es redirigido al login.
- [] GIVEN un usuario autenticado sin el rol requerido, WHEN intenta acceder a una ruta restringida, THEN ve pantalla 403 o es redirigido.
- [] Las rutas publicas (catalogo, login, registro) son accesibles sin autenticacion.

****Notas Tecnicas**:**

- Route guards (React Router / Angular Guards)

- HOC `withAuth(Component, requiredRoles)` o equivalente
- Zustand auth store como source of truth del estado de autenticacion

US-066: Manejo de token expirado en frontend

- **Titulo**: Renovacion transparente de sesion
- **Historia**: Como **Sistema**, quiero interceptar respuestas 401 en el frontend y renovar el token automaticamente, para que el cliente no pierda su sesion por expiracion del access token.
- **Prioridad**: Alta
- **Dependencias**: US-003

Criterios de Aceptacion:

- [] GIVEN un access token expirado, WHEN el frontend recibe 401, THEN automaticamente llama al endpoint de refresh y reintenta la request original.
- [] GIVEN un refresh token tambien expirado, WHEN el refresh falla, THEN se redirige al usuario al login.
- [] El proceso es transparente para el usuario (no ve errores intermitentes).
- [] Si hay multiples requests concurrentes y el token expira, se encolan las requests y se resuelven todas tras el refresh (queue de requests).

Notas Tecnicas:

- Axios/fetch interceptor que detecta 401
- Singleton de refresh en progreso para evitar multiples refresh simultaneos
- Cola de requests pendientes que se resuelven post-refresh

US-067: Manejo de errores global en frontend

- **Titulo**: Gestion centralizada de errores HTTP

- **Historia**: Como **Cliente**, quiero ver mensajes de error claros cuando algo falla, para entender que paso y que puedo hacer.
- **Prioridad**: Media
- **Dependencias**: US-002

Criterios de Aceptacion:

- [] GIVEN un error 400 (validacion), WHEN ocurre, THEN se muestran los errores de campo especificos.
- [] GIVEN un error 403, WHEN ocurre, THEN se muestra "No tenes permisos para esta accion".
- [] GIVEN un error 404, WHEN ocurre, THEN se muestra "Recurso no encontrado".
- [] GIVEN un error 429, WHEN ocurre, THEN se muestra "Demasiadas solicitudes, espera un momento".
- [] GIVEN un error 500, WHEN ocurre, THEN se muestra "Error interno, intenta de nuevo mas tarde".

Notas Tecnicas:

- Error boundary global en React/Angular
- Interceptor de Axios/HttpClient para mapear codigos a mensajes
- Toast/notification system para errores no bloqueantes

EPIC 03 — Gestion de Categorias

US-007: Crear categoria

- **Titulo**: Alta de categoria de productos
- **Historia**: Como **Gestor de Stock**, quiero crear categorias para organizar los productos, para que los clientes encuentren lo que buscan mas facilmente.
- **Prioridad**: Alta
- **Dependencias**: US-006

Criterios de Aceptacion:

- [] GIVEN un Gestor de Stock autenticado, WHEN crea una categoria con nombre y opcionalmente una categoria padre, THEN la categoria se almacena y esta disponible en el catalogo.
- [] GIVEN un nombre de categoria duplicado en el mismo nivel, WHEN intenta crearla, THEN el sistema rechaza con error de validacion.
- [] La categoria puede ser raiz (sin padre) o subcategoria (con `parentId`).
- [] El nombre es obligatorio, no vacio, maximo 100 caracteres.

****Notas Tecnicas**:**

- Endpoint: `POST /api/categorias`
- Tabla: `Categoria` con `id`, `nombre`, `parentId` (self-referencing FK)
- Consultas jerarquicas con CTE recursivo (Common Table Expression)

US-008: Listar categorias jerarquicas

- ****Titulo****: Visualizacion del arbol de categorias
- ****Historia****: Como ****Cliente****, quiero ver las categorias organizadas en forma jerarquica, para navegar el catalogo de manera intuitiva.
- ****Prioridad****: Alta
- ****Dependencias****: US-007

****Criterios de Aceptacion**:**

- [] GIVEN categorias existentes con relaciones padre-hijo, WHEN se solicita el listado, THEN se retorna un arbol anidado con categorias y subcategorias.
- [] Las categorias sin padre aparecen como nodos raiz.
- [] Cada nodo incluye: id, nombre, subcategorias (recursivo).
- [] La respuesta es publica (no requiere autenticacion).

****Notas Tecnicas**:**

- Endpoint: `GET /api/categorias` (publico)
- Query con CTE recursivo para armar el arbol

- Cachear respuesta si el volumen lo justifica

US-009: Editar categoria

- ****Titulo****: Modificacion de categoria existente
- ****Historia****: Como ****Gestor de Stock****, quiero editar el nombre o la jerarquia de una categoria, para mantener el catalogo organizado correctamente.
- ****Prioridad****: Media
- ****Dependencias****: US-007

****Criterios de Aceptacion****:

- [] GIVEN una categoria existente, WHEN se modifica su nombre o parentId, THEN los cambios se persisten y se reflejan en el catalogo.
- [] GIVEN un cambio de parentId que generaria un ciclo (A padre de B, B padre de A), WHEN se intenta, THEN el sistema rechaza con error.
- [] No se puede asignar una categoria como padre de si misma.

****Notas Tecnicas****:

- Endpoint: `PUT /api/categorias/:id`
- Validacion de ciclos con CTE antes de persistir

US-010: Eliminar categoria (soft delete)

- ****Titulo****: Baja logica de categoria
- ****Historia****: Como ****Gestor de Stock****, quiero dar de baja una categoria que ya no se usa, para mantener el catalogo limpio sin perder datos historicos.
- ****Prioridad****: Media
- ****Dependencias****: US-007

****Criterios de Aceptacion**:**

- [] GIVEN una categoria sin productos activos asociados, WHEN se solicita su eliminacion, THEN se marca como eliminada (soft delete) y deja de aparecer en el catalogo publico.
- [] GIVEN una categoria con productos activos, WHEN se intenta eliminar, THEN el sistema rechaza indicando que debe reasignar los productos primero.
- [] Las subcategorias de una categoria eliminada deben reasignarse o eliminarse previamente.

****Notas Tecnicas**:**

- Endpoint: `DELETE /api/categorias/:id`
- Campo `deletedAt` (timestamp nullable) para soft delete
- Filtro global en queries publicas: `WHERE deletedAt IS NULL`

EPIC 04 — Gestion de Ingredientes y Alergenos

US-011: Crear ingrediente

- ****Titulo****: Alta de ingrediente
- ****Historia****: Como ****Gestor de Stock****, quiero registrar ingredientes indicando si son alergenicos, para informar correctamente a los clientes sobre la composicion de los productos.
- ****Prioridad****: Alta
- ****Dependencias****: US-006

****Criterios de Aceptacion**:**

- [] GIVEN un Gestor de Stock autenticado, WHEN crea un ingrediente con nombre y flag de alergeno, THEN el ingrediente queda disponible para asociar a productos.
- [] El campo `esAlergeno` es booleano, obligatorio.
- [] El nombre del ingrediente es unico, no vacio.

****Notas Tecnicas**:**

- Endpoint: `POST /api/ingredientes`
- Tabla: `Ingrediente` con `id`, `nombre`, `esAlergeno`

US-012: Listar ingredientes

- **Titulo**: Consulta de ingredientes
- **Historia**: Como **Gestor de Stock**, quiero ver todos los ingredientes registrados, para gestionar su asociacion con productos.
- **Prioridad**: Alta
- **Dependencias**: US-011

Criterios de Aceptacion:

- [] GIVEN ingredientes existentes, WHEN se solicita el listado, THEN se retornan todos los ingredientes con su flag de alergenico.
- [] Se puede filtrar por `esAlergeno=true` para ver solo alergenicos.
- [] Soporta paginacion.

Notas Tecnicas:

- Endpoint: `GET /api/ingredientes`
- Query params: `?esAlergeno=true&page=1&limit=20`

US-013: Editar ingrediente

- **Titulo**: Modificacion de ingrediente
- **Historia**: Como **Gestor de Stock**, quiero editar un ingrediente existente, para corregir datos o actualizar su clasificacion como alergenico.
- **Prioridad**: Media
- **Dependencias**: US-011

****Criterios de Aceptacion**:**

- [] GIVEN un ingrediente existente, WHEN se modifica nombre o flag de alergeno, THEN los cambios se persisten.
- [] El nuevo nombre no puede coincidir con otro ingrediente existente.

****Notas Tecnicas**:**

- Endpoint: `PUT /api/ingredientes/:id`

US-014: Eliminar ingrediente (soft delete)

- ****Titulo****: Baja logica de ingrediente
- ****Historia****: Como ****Gestor de Stock****, quiero dar de baja un ingrediente, para que no se pueda asociar a nuevos productos sin perder los registros historicos.
- ****Prioridad****: Baja
- ****Dependencias****: US-011

****Criterios de Aceptacion**:**

- [] GIVEN un ingrediente, WHEN se elimina logicamente, THEN deja de aparecer para nuevas asociaciones pero se mantiene en productos existentes.
- [] Soft delete con campo `deletedAt`.

****Notas Tecnicas**:**

- Endpoint: `DELETE /api/ingredientes/:id`

EPIC 05 — Gestion de Productos y Catalogo

US-015: Crear producto

- **Titulo**: Alta de producto en el catalogo
- **Historia**: Como **Gestor de Stock**, quiero dar de alta un producto con su precio, stock, imagen y descripcion, para que los clientes puedan verlo y comprarlo.
- **Prioridad**: Alta
- **Dependencias**: US-007, US-011

Criterios de Aceptacion:

- [] GIVEN un Gestor de Stock autenticado, WHEN crea un producto con nombre, descripcion, precio, stock, imagen y disponibilidad, THEN el producto se persiste y aparece en el catalogo.
- [] El precio se almacena con precision fija (DECIMAL/NUMERIC, no float).
- [] El stock es un entero ≥ 0 .
- [] La disponibilidad ('disponible') es booleano, default 'true'.
- [] Todos los campos obligatorios se validan en backend.

Notas Tecnicas:

- Endpoint: 'POST /api/productos'
- Precio: 'DECIMAL(10,2)' o 'NUMERIC' con precision fija
- Imagen: URL o upload a storage (S3/local)

US-016: Asociar producto a categorias

- **Titulo**: Clasificacion de producto en categorias
- **Historia**: Como **Gestor de Stock**, quiero asociar un producto a una o mas categorias, para que aparezca en las secciones correctas del catalogo.
- **Prioridad**: Alta
- **Dependencias**: US-015, US-007

Criterios de Aceptacion:

- [] GIVEN un producto existente, WHEN se le asignan categorias, THEN aparece al navegar esas categorias.
- [] Un producto puede pertenecer a multiples categorias (M2M).
- [] Al quitar una categoria de un producto, deja de aparecer en esa seccion.

****Notas Tecnicas**:**

- Tabla pivote: `ProductoCategoria` con `productold` + `categoriald`
- Endpoint: `PUT /api/productos/:id/categorias` (body: array de categoryIds)

US-017: Asociar ingredientes a producto

- ****Titulo****: Definicion de composicion del producto
- ****Historia****: Como ****Gestor de Stock****, quiero asociar ingredientes a un producto, para que los clientes conozcan su composicion y alergenicos.
- ****Prioridad****: Alta
- ****Dependencias****: US-015, US-011

****Criterios de Aceptacion**:**

- [] GIVEN un producto existente, WHEN se le asignan ingredientes, THEN se muestran en el detalle del producto.
- [] Los ingredientes marcados como alergeno se destacan visualmente en el frontend.
- [] Relacion M2M entre Producto e Ingrediente.

****Notas Tecnicas**:**

- Tabla pivote: `ProductoIngrediente` con `productold` + `ingredienteld`
- Endpoint: `PUT /api/productos/:id/ingredientes` (body: array de ingredientIds)

US-018: Listar productos del catalogo (publico)

- **Titulo**: Navegacion del catalogo de productos
- **Historia**: Como **Cliente**, quiero ver los productos disponibles con su precio, imagen y disponibilidad, para decidir que comprar.
- **Prioridad**: Alta
- **Dependencias**: US-015

Criterios de Aceptacion:

- [] GIVEN productos existentes y disponibles, WHEN se accede al catalogo, THEN se muestran solo los productos con `disponible=true` y `deletedAt IS NULL`.
- [] Cada producto muestra: nombre, precio, imagen, disponibilidad.
- [] Soporta paginacion con `page` y `limit`.
- [] Soporta filtro por categoria.
- [] Soporta busqueda por nombre (ILIKE o full-text).
- [] El endpoint es publico.

Notas Tecnicas:

- Endpoint: `GET /api/productos` (publico)
- Query params: `?categoria=5&busqueda=pizza&page=1&limit=20`
- Incluir conteo total para paginacion en frontend

US-019: Ver detalle de producto

- **Titulo**: Detalle completo de un producto
- **Historia**: Como **Cliente**, quiero ver el detalle completo de un producto incluyendo ingredientes y alergenicos, para tomar una decision de compra informada.
- **Prioridad**: Alta
- **Dependencias**: US-017

Criterios de Aceptacion:

- [] GIVEN un producto existente y disponible, WHEN se consulta su detalle, THEN se retorna: nombre, descripcion, precio, imagen, stock > 0 (sin revelar cantidad exacta), categorias, ingredientes con flag de alergeno.
- [] GIVEN un producto no disponible o eliminado, WHEN se consulta, THEN retorna 404.
- [] El endpoint es publico.

****Notas Tecnicas**:**

- Endpoint: `GET /api/productos/:id` (publico)
- Join con `ProductoIngrediente` -> `Ingrediente` y `ProductoCategoria` -> `Categoria`

US-020: Editar producto

- ****Titulo****: Modificacion de producto
- ****Historia****: Como ****Gestor de Stock****, quiero editar los datos de un producto existente, para mantener la informacion del catalogo actualizada.
- ****Prioridad****: Alta
- ****Dependencias****: US-015

****Criterios de Aceptacion**:**

- [] GIVEN un producto existente, WHEN se modifican sus campos (nombre, descripcion, precio, imagen, disponibilidad), THEN los cambios se persisten.
- [] El precio se valida: debe ser > 0 y con maximo 2 decimales.
- [] No se permite stock negativo.

****Notas Tecnicas**:**

- Endpoint: `PUT /api/productos/:id`

US-021: Gestionar stock de producto

- **Titulo**: Actualizacion de stock
- **Historia**: Como **Gestor de Stock**, quiero actualizar la cantidad en stock de un producto, para reflejar entradas y salidas de mercaderia.
- **Prioridad**: Alta
- **Dependencias**: US-015

Criterios de Aceptacion:

- [] GIVEN un producto existente, WHEN se actualiza el stock con una cantidad, THEN el nuevo stock se persiste.
- [] El stock resultante nunca puede ser negativo.
- [] Se puede hacer incremento (+N) o seteo absoluto segun endpoint.

Notas Tecnicas:

- Endpoint: `PATCH /api/productos/:id/stock` (body: `{ cantidad: number }`)
- Operacion atomica para evitar race conditions (UPDATE con WHERE)

US-022: Eliminar producto (soft delete)

- **Titulo**: Baja logica de producto
- **Historia**: Como **Gestor de Stock**, quiero dar de baja un producto, para que no aparezca en el catalogo sin perder los datos historicos asociados a pedidos.
- **Prioridad**: Media
- **Dependencias**: US-015

Criterios de Aceptacion:

- [] GIVEN un producto existente, WHEN se elimina logicamente, THEN deja de aparecer en el catalogo publico.
- [] Los pedidos historicos que referencian este producto mantienen su informacion (snapshot).
- [] Soft delete con `deletedAt`.

****Notas Tecnicas**:**

- Endpoint: `DELETE /api/productos/:id`

US-023: Filtrar productos por alergen

- ****Titulo****: Filtrado de productos que contienen alergen
- ****Historia****: Como ****Cliente****, quiero filtrar productos que contengan determinados alergen, para evitar alimentos que me generen reacciones alergicas.
- ****Prioridad****: Media
- ****Dependencias****: US-017

****Criterios de Aceptacion****:

- [] GIVEN el catalogo de productos, WHEN filtro excluyendo alergen especificos (por ingredientId), THEN solo veo productos que NO contienen esos ingredientes.
- [] El filtro acepta multiples ingredientes alergen a excluir.

****Notas Tecnicas****:

- Endpoint: `GET /api/productos?excluirAlergen=1,3,7`
- Query con `NOT EXISTS (SELECT ... FROM ProductoIngrediente WHERE ingredientId IN (...))`

EPIC 06 — Gestion del Perfil del Cliente

US-061: Ver perfil propio

- ****Titulo****: Visualizacion del perfil del cliente
- ****Historia****: Como ****Cliente****, quiero ver los datos de mi perfil, para verificar que mi informacion sea correcta.

- ****Prioridad****: Media
- ****Dependencias****: US-002

****Criterios de Aceptacion****:

- [] GIVEN un cliente autenticado, WHEN accede a su perfil, THEN ve: nombre, email, telefono, fecha de registro.
- [] No puede ver datos de otros usuarios.

****Notas Tecnicas****:

- Endpoint: `GET /api/perfil`
- Datos extraidos del JWT userId

US-062: Editar perfil propio

- ****Titulo****: Modificacion de datos personales
- ****Historia****: Como ****Cliente****, quiero editar mis datos personales (nombre, telefono), para mantener mi informacion actualizada.
- ****Prioridad****: Media
- ****Dependencias****: US-061

****Criterios de Aceptacion****:

- [] GIVEN un cliente autenticado, WHEN modifica su nombre o telefono, THEN los cambios se persisten.
- [] El email NO se puede cambiar (es el identificador).
- [] Validacion de formato de telefono.

****Notas Tecnicas****:

- Endpoint: `PUT /api/perfil`

US-063: Cambiar contraseña

- **Titulo**: Cambio de contraseña
- **Historia**: Como **Cliente**, quiero cambiar mi contraseña, para mantener la seguridad de mi cuenta.
- **Prioridad**: Media
- **Dependencias**: US-002

Criterios de Aceptacion:

- [] GIVEN un cliente autenticado, WHEN envia su contraseña actual y la nueva, THEN si la actual es correcta se actualiza la contraseña.
- [] GIVEN contraseña actual incorrecta, WHEN intenta cambiarla, THEN se rechaza con error.
- [] La nueva contraseña debe cumplir los mismos requisitos que en el registro (minimo 8 caracteres).
- [] Se invalidan todos los refresh tokens existentes (forzar re-login).

Notas Tecnicas:

- Endpoint: `PUT /api/perfil/password`
- Body: `{ passwordActual, passwordNueva }`
- Verificar con `bcrypt.compare` antes de hashear la nueva

EPIC 07 — Gestion de Direcciones de Entrega

US-024: Crear direccion de entrega

- **Titulo**: Alta de direccion de entrega
- **Historia**: Como **Cliente**, quiero agregar direcciones de entrega a mi perfil, para seleccionarlal as al realizar un pedido.
- **Prioridad**: Alta

- **Dependencias**: US-002

Criterios de Aceptacion:

- [] GIVEN un cliente autenticado, WHEN agrega una direccion con calle, numero, piso/depto (opcional), ciudad, codigo postal, THEN la direccion se asocia a su cuenta.
- [] Si es la primera direccion, se marca como determinada automaticamente.
- [] Un cliente puede tener multiples direcciones.

Notas Tecnicas:

- Endpoint: `POST /api/direcciones`
- Tabla: `Direccion` con FK a `Usuario`
- Campo `esPredeterminada` booleano

US-025: Listar direcciones del cliente

- **Titulo**: Consulta de direcciones propias
- **Historia**: Como **Cliente**, quiero ver todas mis direcciones guardadas, para gestionar donde recibo mis pedidos.
- **Prioridad**: Alta
- **Dependencias**: US-024

Criterios de Aceptacion:

- [] GIVEN un cliente autenticado, WHEN solicita sus direcciones, THEN recibe solo las direcciones asociadas a su cuenta.
- [] La direccion determinada se indica claramente.

Notas Tecnicas:

- Endpoint: `GET /api/direcciones`
- Filtrado automatico por `userId` extraido del JWT

US-026: Editar direccion de entrega

- **Titulo**: Modificacion de direccion
- **Historia**: Como **Cliente**, quiero editar una direccion existente, para corregir o actualizar mis datos de entrega.
- **Prioridad**: Media
- **Dependencias**: US-024

Criterios de Aceptacion:

- [] GIVEN una direccion propia, WHEN el cliente la modifica, THEN los cambios se persisten.
- [] Un cliente no puede editar direcciones de otro usuario.

Notas Tecnicas:

- Endpoint: `PUT /api/direcciones/:id`
- Validar ownership: `direccion.userId === jwt.userId`

US-027: Eliminar direccion de entrega

- **Titulo**: Baja de direccion
- **Historia**: Como **Cliente**, quiero eliminar una direccion que ya no uso, para mantener limpio mi listado de direcciones.
- **Prioridad**: Baja
- **Dependencias**: US-024

Criterios de Aceptacion:

- [] GIVEN una direccion propia sin pedidos activos asociados, WHEN solicita eliminarla, THEN se elimina (logica o fisica segun politica).

- [] Si la direccion eliminada era la predeterminada, se debe asignar otra o dejar sin predeterminada.

****Notas Tecnicas**:**

- Endpoint: `DELETE /api/direcciones/:id`

US-028: Establecer direccion predeterminada

- ****Titulo****: Seleccion de direccion predeterminada
- ****Historia****: Como ****Cliente****, quiero marcar una direccion como predeterminada, para que se preseleccione al crear un pedido.
- ****Prioridad****: Media
- ****Dependencias****: US-024

****Criterios de Aceptacion****:

- [] GIVEN un cliente con multiples direcciones, WHEN marca una como predeterminada, THEN la anterior predeterminada pierde ese estado y la nueva lo obtiene.
- [] Solo una direccion puede ser predeterminada a la vez.

****Notas Tecnicas****:

- Endpoint: `PATCH /api/direcciones/:id/predeterminada`
- Transaccion: quitar flag de la anterior, setear en la nueva

EPIC 08 — Carrito de Compras

US-029: Agregar producto al carrito

- ****Titulo****: Agregado de producto al carrito

- **Historia**: Como **Cliente**, quiero agregar productos al carrito indicando cantidad, para ir armando mi pedido.
- **Prioridad**: Alta
- **Dependencias**: US-018

Criterios de Aceptacion:

- [] GIVEN un producto disponible con stock > 0, WHEN el cliente lo agrega al carrito con cantidad, THEN aparece en el carrito con el subtotal calculado.
- [] GIVEN un producto ya en el carrito, WHEN se agrega de nuevo, THEN se incrementa la cantidad.
- [] La cantidad debe ser >= 1.
- [] El carrito persiste al cerrar el navegador (localStorage).

Notas Tecnicas:

- Store: Zustand con middleware `persist` (localStorage)
- Sin endpoint backend — el carrito es client-side only
- Estructura: `{ items: [{ productid, nombre, precio, cantidad, imagen, exclusiones }] }`

US-030: Personalizar producto (exclusion de ingredientes)

- **Titulo**: Customizacion del producto en el carrito
- **Historia**: Como **Cliente**, quiero excluir ingredientes de un producto al agregarlo al carrito, para personalizar mi pedido segun mis preferencias o restricciones alimentarias.
- **Prioridad**: Alta
- **Dependencias**: US-029, US-017

Criterios de Aceptacion:

- [] GIVEN un producto con ingredientes asociados, WHEN el cliente selecciona ingredientes a excluir, THEN se almacenan las exclusiones en el item del carrito.
- [] Solo se pueden excluir ingredientes que el producto efectivamente tiene.
- [] Las exclusiones se muestran claramente en el resumen del carrito.

- [] Las exclusiones se representan como array de IDs de ingredientes.

****Notas Tecnicas**:**

- En el Zustand store: `exclusiones: number[]` (IDs de ingredientes excluidos)
- Al crear el pedido, se envian como `INTEGER[]` al backend
- UI: checkboxes o toggles sobre la lista de ingredientes del producto

US-031: Modificar cantidad de item en el carrito

- ****Titulo****: Ajuste de cantidades en el carrito
- ****Historia****: Como ****Cliente****, quiero cambiar la cantidad de un producto en el carrito, para ajustar mi pedido antes de confirmarlo.
- ****Prioridad****: Alta
- ****Dependencias****: US-029

****Criterios de Aceptacion**:**

- [] GIVEN un item en el carrito, WHEN se cambia la cantidad a un valor ≥ 1 , THEN se actualiza el subtotal.
- [] GIVEN un item en el carrito, WHEN se pone cantidad 0, THEN se elimina del carrito.
- [] Los cambios se persisten inmediatamente en localStorage.

****Notas Tecnicas**:**

- Zustand actions: `updateQuantity(productId, newQty)`
- Recalcular totales reactivamente

US-032: Eliminar item del carrito

- ****Titulo****: Remocion de producto del carrito

- **Historia**: Como **Cliente**, quiero quitar un producto del carrito, para descartar algo que ya no quiero pedir.
- **Prioridad**: Alta
- **Dependencias**: US-029

Criterios de Aceptacion:

- [] GIVEN un item en el carrito, WHEN se elimina, THEN desaparece del carrito y se recalcula el total.
- [] El cambio persiste en localStorage.

Notas Tecnicas:

- Zustand action: `removeItem(productId)`

US-033: Ver resumen del carrito

- **Titulo**: Visualizacion del carrito
- **Historia**: Como **Cliente**, quiero ver un resumen del carrito con todos los productos, cantidades, exclusiones y el total, para revisar mi pedido antes de confirmarlo.
- **Prioridad**: Alta
- **Dependencias**: US-029

Criterios de Aceptacion:

- [] GIVEN items en el carrito, WHEN el cliente accede al carrito, THEN ve: nombre del producto, cantidad, precio unitario, exclusiones, subtotal por item, y total general.
- [] Si el carrito esta vacio, se muestra un mensaje indicandolo con link al catalogo.
- [] Los precios se muestran con 2 decimales.

Notas Tecnicas:

- Componente: `CartSummary`
- Selector derivado de Zustand para el total: `useCartStore(state => state.total())`

US-034: Vaciar carrito

- ****Titulo****: Limpieza completa del carrito
- ****Historia****: Como ****Cliente****, quiero vaciar el carrito de una vez, para empezar de cero si cambie de opinion.
- ****Prioridad****: Baja
- ****Dependencias****: US-029

****Criterios de Aceptacion****:

- [] GIVEN un carrito con items, WHEN el cliente elige vaciar, THEN se confirma la accion con un dialogo y se eliminan todos los items.
- [] El total pasa a \$0.

****Notas Tecnicas****:

- Zustand action: `clearCart()`
- UI: boton con confirmacion modal

EPIC 09 — Validaciones Pre-Checkout

US-069: Validar disponibilidad al hacer checkout

- ****Titulo****: Verificacion de stock antes de crear el pedido
- ****Historia****: Como ****Sistema****, quiero validar la disponibilidad de cada producto del carrito antes de crear el pedido, para evitar que el cliente compre algo que ya no esta disponible.
- ****Prioridad****: Alta
- ****Dependencias****: US-029, US-035

****Criterios de Aceptacion**:**

- [] GIVEN items en el carrito, WHEN el cliente inicia el checkout, THEN se verifica que cada producto siga disponible y con stock suficiente.
- [] GIVEN un producto sin stock suficiente, WHEN se detecta, THEN se notifica al cliente indicando que producto y cuanto stock queda.
- [] GIVEN un producto que fue eliminado o desactivado, WHEN se detecta, THEN se notifica al cliente y se sugiere eliminarlo del carrito.

****Notas Tecnicas**:**

- Endpoint: `POST /api/pedidos/validar` o validacion dentro de `POST /api/pedidos`
- Pre-validacion client-side + validacion definitiva server-side en la transaccion

US-070: Verificar precios actualizados al hacer checkout

- ****Titulo****: Deteccion de cambios de precio antes de pagar
- ****Historia****: Como ****Cliente****, quiero ser notificado si un precio cambio desde que agregue el producto al carrito, para decidir si sigo con la compra al nuevo precio.
- ****Prioridad****: Media
- ****Dependencias****: US-069

****Criterios de Aceptacion**:**

- [] GIVEN un producto en el carrito cuyo precio cambio desde que fue agregado, WHEN se inicia el checkout, THEN se notifica al cliente mostrando precio viejo vs. nuevo.
- [] El cliente puede aceptar el nuevo precio y continuar, o cancelar.

****Notas Tecnicas**:**

- Comparar `cart.item.precio` (localStorage) vs `producto.precio` (DB) al validar
- Response incluye array de cambios detectados

US-035: Crear pedido desde el carrito

- **Titulo**: Creacion de pedido
- **Historia**: Como **Cliente**, quiero confirmar mi carrito y crear un pedido, para proceder al pago y recibir mis productos.
- **Prioridad**: Alta
- **Dependencias**: US-029, US-024

Criterios de Aceptacion:

- [] GIVEN un carrito con items y una direccion seleccionada, WHEN el cliente confirma el pedido, THEN se crea un pedido en estado PENDIENTE.
- [] Se genera un snapshot del precio de cada producto al momento de la creacion (RN-PE02).
- [] Se genera un snapshot de la direccion de entrega seleccionada (RN-PE03).
- [] La creacion es atomica: si falla cualquier parte, no se persiste nada (Unit of Work).
- [] Las exclusiones de ingredientes se almacenan como `INTEGER[]` en cada linea del pedido.
- [] Se vacia el carrito tras la creacion exitosa.
- [] Se registra la entrada inicial en `HistorialEstadoPedido` con estado PENDIENTE (RN-FS07).

Reglas de Negocio: RN-PE01, RN-PE02 (precio snapshot), RN-PE03 (direccion snapshot), RN-PE04, RN-PE05, RN-PE06, RN-PE07, RN-PE08, RN-FS07 (auditoria).

Notas Tecnicas:

- Endpoint: `POST /api/pedidos`
- Body: `{ direccionId, items: [{ productId, cantidad, exclusiones: number[] }] }`
- Patron: Unit of Work — transaccion con INSERT en `Pedido`, `DetallePedido[]`, `HistorialEstadoPedido`
- Snapshot: copiar precio actual y datos de direccion a campos del pedido, no FK directa
- Validar stock disponible DENTRO de la transaccion (SELECT FOR UPDATE)

US-036: Validacion de stock al crear pedido

- **Titulo**: Verificacion de disponibilidad al confirmar
- **Historia**: Como **Sistema**, quiero validar que haya stock suficiente de cada producto al crear un pedido, para evitar ventas de productos agotados.
- **Prioridad**: Alta
- **Dependencias**: US-035

Criterios de Aceptacion:

- [] GIVEN un item del pedido con cantidad 3 y stock disponible 2, WHEN se intenta crear el pedido, THEN se rechaza con error indicando el producto y stock disponible.
- [] La verificacion se hace de forma atomica dentro de la transaccion (SELECT FOR UPDATE).
- [] Si algun producto no tiene stock suficiente, no se crea NINGUN item del pedido.

Reglas de Negocio: RN-PE04, RN-PE05 (stock atomico dentro de transaccion).

Notas Tecnicas:

- `SELECT stock FROM Producto WHERE id = ? FOR UPDATE` dentro de la transaccion
- Validar TODOS los items antes de hacer cualquier INSERT

US-037: Snapshot de precios en el pedido

- **Titulo**: Captura de precios al momento de la compra
- **Historia**: Como **Sistema**, quiero almacenar el precio de cada producto al momento de crear el pedido, para que cambios futuros de precios no afecten pedidos existentes.
- **Prioridad**: Alta
- **Dependencias**: US-035

Criterios de Aceptacion:

- [] GIVEN un pedido creado, WHEN el precio del producto cambia posteriormente, THEN el pedido mantiene el precio original de la compra.
- [] El precio snapshot se almacena en `DetallePedido.precioUnitario`.
- [] El total del pedido se calcula a partir de los precios snapshot.

****Reglas de Negocio**:** RN-PE02, RN-DA06.

****Notas Tecnicas**:**

- Campo `precioUnitario DECIMAL(10,2)` en `DetallePedido`
- Campo `total DECIMAL(10,2)` calculado y almacenado en `Pedido`

US-038: Snapshot de direccion en el pedido

- ****Titulo**:** Captura de direccion al momento de la compra
- ****Historia**:** Como ****Sistema****, quiero almacenar la direccion de entrega al momento de crear el pedido, para que modificaciones futuras de la direccion no afecten pedidos en curso.
- ****Prioridad**:** Alta
- ****Dependencias**:** US-035

****Criterios de Aceptacion**:**

- [] GIVEN un pedido creado, WHEN el cliente modifica la direccion original, THEN el pedido mantiene la direccion snapshot.
- [] Los campos de direccion se copian directamente al pedido o a una tabla de snapshot.

****Reglas de Negocio**:** RN-PE03, RN-DA06.

****Notas Tecnicas**:**

- Campos en `Pedido`: `direccionCalle`, `direccionNumero`, `direccionPiso`, `direccionCiudad`, `direccionCP`
- Alternativa: JSON serializado en campo `direccionSnapshot`

EPIC 11 — Pagos con MercadoPago

US-045: Iniciar proceso de pago

- **Titulo**: Creacion de orden de pago en MercadoPago
- **Historia**: Como **Cliente**, quiero pagar mi pedido a traves de MercadoPago, para completar la compra de forma segura.
- **Prioridad**: Alta
- **Dependencias**: US-035

Criterios de Aceptacion:

- [] GIVEN un pedido en estado PENDIENTE, WHEN el cliente inicia el pago, THEN se crea una orden de pago en MercadoPago via Orders API.
- [] Se genera y almacena un idempotency key para evitar pagos duplicados.
- [] El cliente es redirigido al checkout de MercadoPago o se le muestra el formulario embebido.
- [] Los datos de tarjeta se tokenizan en el browser (PCI SAQ-A: nunca tocan nuestro servidor).

Reglas de Negocio: RN-PA01, RN-PA02, RN-PA09, RN-AU09.

Notas Tecnicas:

- Endpoint: `POST /api/pagos/crear` (body: `{ pedidoId }`)
- MercadoPago Orders API para crear la preferencia/orden
- Idempotency key: UUID almacenado en tabla `Pago` asociado al `pedidoId`
- Tokenizacion con SDK JS de MercadoPago en el frontend

US-046: Procesar webhook de pago (IPN)

- ****Titulo****: Recepcion y procesamiento de notificaciones de MercadoPago
- ****Historia****: Como ****Sistema****, quiero procesar las notificaciones IPN de MercadoPago, para actualizar el estado del pedido segun el resultado del pago.
- ****Prioridad****: Alta
- ****Dependencias****: US-045

****Criterios de Aceptacion****:

- [] GIVEN una notificacion IPN con status `approved`, WHEN se procesa, THEN el pedido pasa de PENDIENTE a CONFIRMADO.
- [] GIVEN una notificacion con status `rejected`, WHEN se procesa, THEN se marca el pago como rechazado y el pedido permanece en PENDIENTE.
- [] GIVEN una notificacion con status `pending` o `in\_process`, WHEN se procesa, THEN se actualiza el estado del pago pero el pedido sigue PENDIENTE.
- [] GIVEN una notificacion con status `cancelled`, WHEN se procesa, THEN se registra y el pedido puede ser cancelado.
- [] El webhook responde 200 inmediatamente y procesa asincronicamente.
- [] Se valida la firma/origen de la notificacion.
- [] El procesamiento es idempotente: recibir la misma notificacion 2 veces no causa efectos duplicados.

****Reglas de Negocio****: RN-PA02, RN-PA03, RN-PA04, RN-PA05, RN-PA06, RN-PA07.

****Notas Tecnicas****:

- Endpoint: `POST /api/webhooks/mercadopago`
- Verificar header de firma de MercadoPago
- Idempotencia: verificar si la notificacion ya fue procesada antes de actuar
- Responder 200 OK antes de procesar (o usar cola)

US-047: Consultar estado de pago

- ****Titulo****: Verificacion del estado de pago de un pedido
- ****Historia****: Como ****Cliente****, quiero ver el estado de pago de mi pedido, para saber si el pago fue procesado correctamente.
- ****Prioridad****: Media
- ****Dependencias****: US-045

****Criterios de Aceptacion****:

- [] GIVEN un pedido propio, WHEN consulto su estado de pago, THEN veo: estado (aprobado, pendiente, rechazado, en_proceso, cancelado), monto, fecha de ultimo update.
- [] Solo puedo ver el estado de pago de mis propios pedidos.

****Notas Tecnicas****:

- Endpoint: `GET /api/pedidos/:id/pago`
- Tabla `Pago` con: `id`, `pedidoId`, `mercadoPagoOrderId`, `estado`, `monto`, `idempotencyKey`, `createdAt`, `updatedAt`

US-048: Reintentar pago rechazado

- ****Titulo****: Reintento de pago tras rechazo
- ****Historia****: Como ****Cliente****, quiero poder reintentar el pago si fue rechazado, para completar mi compra sin tener que crear un nuevo pedido.
- ****Prioridad****: Media
- ****Dependencias****: US-046

****Criterios de Aceptacion****:

- [] GIVEN un pedido en PENDIENTE con pago rechazado, WHEN el cliente reintentar, THEN se genera una nueva orden de pago con nuevo idempotency key.
- [] El pedido debe seguir en estado PENDIENTE para permitir reintento.
- [] Se mantiene el registro del intento anterior en la tabla de pagos.

****Notas Tecnicas****:

- Endpoint: `POST /api/pagos/crear` (misma logica, nuevo idempotency key)
- Relacion 1:N entre Pedido y Pago (multiples intentos)

EPIC 12 — Maquina de Estados de Pedidos (FSM)

US-039: Transicion de estado — PENDIENTE a CONFIRMADO

- ****Titulo****: Confirmacion del pedido tras pago aprobado
- ****Historia****: Como ****Sistema****, quiero que el pedido pase automaticamente de PENDIENTE a CONFIRMADO cuando el pago es aprobado, para iniciar su preparacion.
- ****Prioridad****: Alta
- ****Dependencias****: US-035, US-046 (pagos)

****Criterios de Aceptacion****:

- [] GIVEN un pedido en estado PENDIENTE, WHEN se recibe webhook de pago aprobado, THEN el estado cambia a CONFIRMADO.
- [] Se descuenta el stock atomicamente al confirmar (RN-FS03).
- [] Se registra en HistorialEstadoPedido: estado anterior, estado nuevo, timestamp, actor (SISTEMA) (RN-FS07).
- [] La transicion PENDIENTE -> CONFIRMADO solo ocurre por pago aprobado, no manualmente.

****Reglas de Negocio****: RN-FS01 (FSM estricta), RN-FS02, RN-FS03 (decremento atómico), RN-FS04, RN-FS07 (auditoria), RN-FS09.

****Notas Tecnicas****:

- Patron State Machine: validar transicion contra mapa de transiciones permitidas
- `{ PENDIENTE: ['CONFIRMADO', 'CANCELADO'], CONFIRMADO: ['EN\_PREPARACION', 'CANCELADO'], ... }`
- Stock: `UPDATE Producto SET stock = stock - :cant WHERE id = :id AND stock >= :cant`

US-040: Transicion — CONFIRMADO a EN_PREPARACION

- ****Titulo****: Inicio de preparacion del pedido
- ****Historia****: Como ****Gestor de Pedidos****, quiero marcar un pedido confirmado como en preparacion, para que el equipo de cocina comience a trabajar en el.
- ****Prioridad****: Alta
- ****Dependencias****: US-039

****Criterios de Aceptacion****:

- [] GIVEN un pedido en estado CONFIRMADO, WHEN el Gestor de Pedidos avanza el estado, THEN pasa a EN_PREPARACION.
- [] Se registra en HistorialEstadoPedido con el usuario que realizo la accion (RN-FS07).
- [] GIVEN un pedido en cualquier otro estado, WHEN se intenta pasar a EN_PREPARACION, THEN se rechaza con error (RN-FS01).

****Reglas de Negocio****: RN-FS01, RN-FS07, RN-FS09.

****Notas Tecnicas****:

- Endpoint: `PATCH /api/pedidos/:id/estado` (body: `{ nuevoEstado: 'EN\_PREPARACION' }`)
- Validar transicion contra FSM antes de persistir

US-041: Transicion — EN_PREPARACION a EN_CAMINO

- ****Titulo****: Despacho del pedido
- ****Historia****: Como ****Gestor de Pedidos****, quiero marcar un pedido como en camino, para indicar que fue despachado para entrega.
- ****Prioridad****: Alta
- ****Dependencias****: US-040

****Criterios de Aceptacion**:**

- [] GIVEN un pedido en EN_PREPARACION, WHEN se avanza el estado, THEN pasa a EN_CAMINO.
- [] Se registra en HistorialEstadoPedido (RN-FS07).
- [] La transicion solo es valida desde EN_PREPARACION (RN-FS01).

****Reglas de Negocio**:** RN-FS01, RN-FS07, RN-FS09.

****Notas Tecnicas**:**

- Mismo endpoint `PATCH /api/pedidos/:id/estado`

US-042: Transicion — EN_CAMINO a ENTREGADO

- ****Titulo**:** Entrega del pedido
- ****Historia**:** Como ****Gestor de Pedidos****, quiero marcar un pedido como entregado, para cerrar su ciclo de vida.
- ****Prioridad**:** Alta
- ****Dependencias**:** US-041

****Criterios de Aceptacion**:**

- [] GIVEN un pedido en EN_CAMINO, WHEN se marca como entregado, THEN pasa a ENTREGADO.
- [] Se registra en HistorialEstadoPedido (RN-FS07).
- [] Un pedido en estado ENTREGADO no puede cambiar a ningun otro estado (RN-FS06).

****Reglas de Negocio**:** RN-FS01, RN-FS06, RN-FS07, RN-FS09.

****Notas Tecnicas**:**

- Mismo endpoint `PATCH /api/pedidos/:id/estado`

- ENTREGADO es estado terminal

US-043: Cancelar pedido

- ****Titulo****: Cancelacion de pedido

- ****Historia****: Como ****Gestor de Pedidos****, quiero cancelar un pedido en estado PENDIENTE o CONFIRMADO, para gestionar pedidos que no se van a completar.

- ****Prioridad****: Alta

- ****Dependencias****: US-035

****Criterios de Aceptacion****:

- [] GIVEN un pedido en estado PENDIENTE, WHEN se cancela, THEN pasa a CANCELADO (RN-FS08).

- [] GIVEN un pedido en estado CONFIRMADO, WHEN se cancela, THEN pasa a CANCELADO y se devuelve el stock descontado (RN-FS08, RN-FS05).

- [] GIVEN un pedido en EN_PREPARACION, EN_CAMINO o ENTREGADO, WHEN se intenta cancelar, THEN se rechaza con error (RN-FS08, RN-RB08).

- [] Se registra en HistorialEstadoPedido el motivo de cancelacion (RN-FS07).

- [] CANCELADO es estado terminal (RN-FS06).

****Reglas de Negocio****: RN-FS01, RN-FS05 (restauracion de stock), RN-FS06, RN-FS07, RN-FS08, RN-FS09, RN-RB08.

****Notas Tecnicas****:

- Endpoint: `PATCH /api/pedidos/:id/estado` (body: `{ nuevoEstado: 'CANCELADO', motivo: '...' }`)

- Si venia de CONFIRMADO: `UPDATE Producto SET stock = stock + :cant WHERE id = :id`

- Transaccion atomica: cambio de estado + devolucion de stock

US-044: Auditoria de cambios de estado

- **Titulo**: Historial de estados del pedido
- **Historia**: Como **Admin**, quiero ver el historial completo de estados de un pedido, para auditar su procesamiento y resolver incidentes.
- **Prioridad**: Alta
- **Dependencias**: US-039

Criterios de Aceptacion:

- [] GIVEN un pedido, WHEN se consulta su historial, THEN se retorna una lista cronologica de todas las transiciones con: estado anterior, estado nuevo, fecha/hora, usuario/sistema que realizo el cambio.
- [] El historial es append-only: no se pueden editar ni eliminar registros.
- [] Cada transicion incluye motivo (obligatorio para cancelaciones).

Reglas de Negocio: RN-FS07, RN-FS09, RN-DA05.

Notas Tecnicas:

- Endpoint: `GET /api/pedidos/:id/historial`
- Tabla: `HistorialEstadoPedido` con `id`, `pedidoId`, `estadoAnterior`, `estadoNuevo`, `timestamp`, `usuarioId`, `motivo`
- Sin UPDATE ni DELETE en esta tabla (append-only)

EPIC 13 — Visualizacion de Pedidos

US-049: Ver mis pedidos (Cliente)

- **Titulo**: Historial de pedidos del cliente
- **Historia**: Como **Cliente**, quiero ver el listado de todos mis pedidos con su estado actual, para hacer seguimiento de mis compras.
- **Prioridad**: Alta

- **Dependencias**: US-035

Criterios de Aceptacion:

- [] GIVEN un cliente autenticado, WHEN consulta sus pedidos, THEN ve una lista paginada con: numero de pedido, fecha, estado actual, total, cantidad de items.
- [] Solo ve sus propios pedidos.
- [] Ordenados por fecha descendente (mas recientes primero).
- [] Soporta filtro por estado.

Notas Tecnicas:

- Endpoint: `GET /api/pedidos` (filtrado automatico por userId del JWT)
- Query params: `?estado=EN\_CAMINO&page=1&limit=10`

US-050: Ver detalle de mi pedido (Cliente)

- **Titulo**: Detalle completo de un pedido propio
- **Historia**: Como **Cliente**, quiero ver el detalle completo de uno de mis pedidos, para conocer los productos, cantidades, exclusiones, direccion y estado de pago.
- **Prioridad**: Alta
- **Dependencias**: US-049

Criterios de Aceptacion:

- [] GIVEN un pedido propio, WHEN consulto su detalle, THEN veo: items (nombre, cantidad, precio snapshot, exclusiones), direccion snapshot, estado actual, total, estado de pago.
- [] No puedo ver pedidos de otros clientes (403).

Notas Tecnicas:

- Endpoint: `GET /api/pedidos/:id`
- Join con `DetallePedido` para los items

US-051: Ver todos los pedidos (Gestor de Pedidos)

- ****Titulo****: Panel de gestion de pedidos
- ****Historia****: Como ****Gestor de Pedidos****, quiero ver todos los pedidos del sistema con filtros por estado, para gestionar el flujo de preparacion y entrega.
- ****Prioridad****: Alta
- ****Dependencias****: US-035

****Criterios de Aceptacion****:

- [] GIVEN un Gestor de Pedidos autenticado, WHEN accede al panel, THEN ve todos los pedidos de todos los clientes.
- [] Puede filtrar por estado (especialmente CONFIRMADO, EN_PREPARACION, EN_CAMINO).
- [] Puede filtrar por rango de fechas.
- [] Puede buscar por numero de pedido o nombre de cliente.
- [] Paginacion obligatoria.

****Notas Tecnicas****:

- Endpoint: `GET /api/admin/pedidos` (rol PEDIDOS o ADMIN)
- Query params:
`?estado=CONFIRMADO&desde=2026-01-01&hasta=2026-03-31&page=1&limit=20`

US-052: Ver detalle de cualquier pedido (Gestor/Admin)

- ****Titulo****: Detalle completo de pedido para gestion
- ****Historia****: Como ****Gestor de Pedidos****, quiero ver el detalle completo de cualquier pedido, para tomar decisiones sobre su procesamiento.
- ****Prioridad****: Alta
- ****Dependencias****: US-051

****Criterios de Aceptacion**:**

- [] GIVEN un pedido del sistema, WHEN el gestor consulta su detalle, THEN ve: items con snapshots, direccion snapshot, historial de estados, datos del cliente, estado de pago.
- [] Incluye el historial completo de estados con fechas y actores.

****Notas Tecnicas**:**

- Endpoint: `GET /api/admin/pedidos/:id`
- Joins: DetallePedido + HistorialEstadoPedido + Pago

EPIC 14 — Notificaciones y Feedback UX

US-071: Confirmacion de pedido creado

- ****Titulo****: Feedback visual al crear pedido
- ****Historia****: Como ****Cliente****, quiero recibir una confirmacion visual clara cuando mi pedido se crea exitosamente, para saber que todo salio bien.
- ****Prioridad****: Media
- ****Dependencias****: US-035

****Criterios de Aceptacion**:**

- [] GIVEN un pedido creado exitosamente, WHEN se completa la transaccion, THEN se muestra pantalla de confirmacion con: numero de pedido, resumen de items, total, direccion, y estado "PENDIENTE - Esperando pago".
- [] Se incluye un boton/link para ir a pagar.
- [] Se incluye un boton para ver el detalle del pedido.

****Notas Tecnicas**:**

- Componente: `OrderConfirmation`
- Redirigir automaticamente post-creacion

US-072: Feedback de estado de pago

- ****Titulo****: Retorno de MercadoPago al sitio
- ****Historia****: Como ****Cliente****, quiero ver el resultado de mi pago al volver de MercadoPago, para saber si debo reintentar o si el pago fue exitoso.
- ****Prioridad****: Alta
- ****Dependencias****: US-045

****Criterios de Aceptacion****:

- [] GIVEN un pago exitoso (callback con status=approved), WHEN el cliente vuelve al sitio, THEN ve mensaje de exito con estado actualizado del pedido.
- [] GIVEN un pago rechazado, WHEN vuelve al sitio, THEN ve mensaje de rechazo con opcion de reintentar.
- [] GIVEN un pago pendiente, WHEN vuelve, THEN ve mensaje de que esta en proceso con indicacion de esperar.

****Notas Tecnicas****:

- URLs de callback de MercadoPago: `success\_url`, `failure\_url`, `pending\_url`
- Pagina de retorno que consulta el estado actual del pedido/pago

EPIC 15 — Administracion de Usuarios

US-053: Listar usuarios del sistema

- ****Titulo****: Panel de usuarios
- ****Historia****: Como ****Admin****, quiero ver todos los usuarios registrados con su rol y estado, para gestionar el acceso al sistema.
- ****Prioridad****: Alta

- **Dependencias**: US-005

Criterios de Aceptacion:

- [] GIVEN un Admin autenticado, WHEN accede al listado de usuarios, THEN ve: nombre, email, rol, fecha de registro, estado (activo/inactivo).
- [] Soporta busqueda por nombre o email.
- [] Soporta filtro por rol.
- [] Paginacion obligatoria.

Notas Tecnicas:

- Endpoint: `GET /api/admin/usuarios`
- Solo accesible con rol ADMIN

US-054: Editar usuario (Admin)

- **Titulo**: Modificacion de datos de usuario
- **Historia**: Como **Admin**, quiero editar los datos y rol de cualquier usuario, para corregir informacion o ajustar permisos.
- **Prioridad**: Media
- **Dependencias**: US-053

Criterios de Aceptacion:

- [] GIVEN un Admin, WHEN edita el rol de un usuario, THEN el cambio se aplica inmediatamente (el proximo token que obtenga ese usuario tendra el nuevo rol).
- [] Un Admin no puede degradar al ultimo ADMIN del sistema.
- [] Se puede activar/desactivar usuarios.

Notas Tecnicas:

- Endpoint: `PUT /api/admin/usuarios/:id`
- Invalidar refresh tokens del usuario modificado para forzar re-login con nuevo rol

US-055: Desactivar usuario

- **Titulo**: Baja logica de usuario
- **Historia**: Como **Admin**, quiero desactivar un usuario, para impedir su acceso sin eliminar sus datos historicos.
- **Prioridad**: Media
- **Dependencias**: US-053

Criterios de Aceptacion:

- [] GIVEN un usuario activo, WHEN el Admin lo desactiva, THEN no puede loguearse mas.
- [] Los pedidos historicos del usuario se mantienen intactos.
- [] Se invalidan todos los refresh tokens del usuario desactivado.

Notas Tecnicas:

- Campo `activo` booleano en `Usuario`
- Validar en login: si `activo=false`, retornar 403 "Cuenta desactivada"
- Endpoint: `PATCH /api/admin/usuarios/:id/estado`

EPIC 16 — Gestion Avanzada de Catalogo (Admin)

US-064: Gestion completa de catalogo (Admin)

- **Titulo**: CRUD de catalogo con privilegios de Admin
- **Historia**: Como **Admin**, quiero tener acceso completo a la gestion del catalogo (productos, categorias, ingredientes), para intervenir cuando sea necesario sin depender del Gestor de Stock.
- **Prioridad**: Media

- **Dependencias**: US-015, US-007, US-011

Criterios de Aceptacion:

- [] GIVEN un Admin, WHEN accede a los endpoints de gestion de catalogo, THEN tiene los mismos permisos que el Gestor de Stock.
- [] El Admin puede crear, editar y eliminar productos, categorias e ingredientes.
- [] Los endpoints de catalogo aceptan tanto rol ADMIN como STOCK.

Notas Tecnicas:

- Guard/middleware: `@Roles('ADMIN', 'STOCK')` en endpoints de gestion de catalogo

US-065: Gestion completa de pedidos (Admin)

- **Titulo**: Control total sobre pedidos
- **Historia**: Como **Admin**, quiero poder gestionar cualquier pedido (ver, avanzar estado, cancelar), para resolver situaciones excepcionales.
- **Prioridad**: Media
- **Dependencias**: US-051, US-043

Criterios de Aceptacion:

- [] GIVEN un Admin, WHEN accede a los endpoints de gestion de pedidos, THEN tiene los mismos permisos que el Gestor de Pedidos.
- [] Los endpoints de pedidos aceptan tanto rol ADMIN como PEDIDOS.

Notas Tecnicas:

- Guard/middleware: `@Roles('ADMIN', 'PEDIDOS')` en endpoints de gestion de pedidos

US-056: Dashboard de metricas generales

- **Titulo**: Panel de metricas del sistema
- **Historia**: Como **Admin**, quiero ver metricas generales del sistema (ventas, pedidos, usuarios), para tomar decisiones informadas sobre el negocio.
- **Prioridad**: Media
- **Dependencias**: US-035, US-053

Criterios de Aceptacion:

- [] GIVEN un Admin autenticado, WHEN accede al dashboard, THEN ve: total de ventas del periodo, cantidad de pedidos por estado, cantidad de usuarios registrados, productos mas vendidos.
- [] Soporta filtro por rango de fechas.
- [] Los datos se actualizan al cambiar el filtro.

Notas Tecnicas:

- Endpoint: `GET /api/admin/metricas/resumen`
- Queries de agregacion: `SUM`, `COUNT`, `GROUP BY`
- Frontend: recharts para visualizaciones

US-057: Grafico de ventas por periodo

- **Titulo**: Visualizacion de evolucion de ventas
- **Historia**: Como **Admin**, quiero ver un grafico de evolucion de ventas por dia/semana/mes, para entender las tendencias del negocio.
- **Prioridad**: Media
- **Dependencias**: US-056

Criterios de Aceptacion:

- [] GIVEN datos de ventas, WHEN selecciono un periodo y granularidad (dia/semana/mes), THEN veo un grafico de lineas con la evolucion de ventas.
- [] El grafico muestra monto total y cantidad de pedidos.

****Notas Tecnicas**:**

- Endpoint: `GET /api/admin/metricas/ventas?desde=...&hasta=...&granularidad=dia`
- Frontend: `` de recharts
- Query con `DATE\_TRUNC` para agrupar por granularidad

US-058: Top productos mas vendidos

- ****Titulo****: Ranking de productos
- ****Historia****: Como ****Admin****, quiero ver el ranking de productos mas vendidos, para entender que productos tienen mayor demanda.
- ****Prioridad****: Media
- ****Dependencias****: US-056

****Criterios de Aceptacion**:**

- [] GIVEN pedidos entregados, WHEN consulto el ranking, THEN veo los top N productos ordenados por cantidad vendida.
- [] Soporta filtro por rango de fechas.
- [] Muestra: nombre del producto, cantidad total vendida, ingreso total generado.

****Notas Tecnicas**:**

- Endpoint: `GET /api/admin/metricas/productos-top?top=10&desde=...&hasta=...`
- Frontend: `` de recharts

US-059: Metricas de pedidos por estado

- **Titulo**: Distribucion de pedidos por estado
- **Historia**: Como **Admin**, quiero ver la distribucion de pedidos por estado, para identificar cuellos de botella en el proceso.
- **Prioridad**: Media
- **Dependencias**: US-056

Criterios de Aceptacion:

- [] GIVEN pedidos existentes, WHEN consulto la distribucion, THEN veo un grafico de torta/barras con la cantidad de pedidos en cada estado.
- [] Soporta filtro por rango de fechas.

Notas Tecnicas:

- Endpoint: `GET /api/admin/metricas/pedidos-por-estado`
- Frontend: `` de recharts

EPIC 18 — Configuración del Sistema

US-060: Configuración del sistema

- **Titulo**: Panel de configuración general
- **Historia**: Como **Admin**, quiero gestionar configuraciones generales del sistema, para ajustar parametros operativos sin tocar código.
- **Prioridad**: Baja
- **Dependencias**: US-006

Criterios de Aceptacion:

- [] GIVEN un Admin, WHEN accede a la configuración, THEN puede ver y modificar parametros como: horarios de atención, zona de entrega, mensajes del sistema.
- [] Los cambios se aplican inmediatamente sin reiniciar el sistema.

- [] Se registra quien modifico que y cuando.

****Notas Tecnicas**:**

- Endpoint: `GET/PUT /api/admin/configuracion`

- Tabla key-value: `Configuracion` con `clave`, `valor`, `updatedBy`, `updatedAt`

Resumen de Historias por Epica

| Epica | Historias | Prioridad general |

| ---- | ----- | ----- |

| 00 - Infraestructura y Setup | US-000, US-000a, US-000b, US-000c, US-000d, US-000e, US-068, US-074 | Alta |

| 01 - Auth y Autorizacion | US-001 a US-006, US-073 | Alta |

| 02 - Navegacion y Layout Base | US-075, US-076, US-066, US-067 | Alta |

| 03 - Categorias | US-007 a US-010 | Alta |

| 04 - Ingredientes y Alergenos | US-011 a US-014 | Alta |

| 05 - Productos y Catalogo | US-015 a US-023 | Alta |

| 06 - Perfil del Cliente | US-061, US-062, US-063 | Media |

| 07 - Direcciones de Entrega | US-024 a US-028 | Alta |

| 08 - Carrito de Compras | US-029 a US-034 | Alta |

| 09 - Validaciones Pre-Checkout | US-069, US-070 | Alta/Media |

| 10 - Creacion de Pedidos | US-035 a US-038 | Alta |

| 11 - Pagos MercadoPago | US-045 a US-048 | Alta |

| 12 - FSM de Pedidos | US-039 a US-044 | Alta |

| 13 - Visualizacion de Pedidos | US-049 a US-052 | Alta |

| 14 - Notificaciones y Feedback UX | US-071, US-072 | Media/Alta |

| 15 - Admin Usuarios | US-053 a US-055 | Alta/Media |

| 16 - Catalogo Admin | US-064, US-065 | Media |

| 17 - Metricas y Dashboard | US-056 a US-059 | Media |

| 18 - Configuracion del Sistema | US-060 | Baja |

****Total: 77 historias de usuario**** (US-000 a US-076) organizadas en 19 epicas, ordenadas por dependencia logica de implementacion.

Orden de Implementacion Recomendado (Plan de Sprints)

****Sprint 0**:** EPIC 00 — Infraestructura y Setup

> Scaffolding monorepo, backend FastAPI, PostgreSQL + Alembic + seed, frontend React+Vite, patrones base, stores Zustand, manejo de errores backend (RFC 7807), validacion y sanitizacion de inputs.

****Sprint 1**:** EPIC 01 (Auth y Autorizacion) + EPIC 02 (Navegacion y Layout Base)

> Registro, login, refresh, logout, RBAC, proteccion de rutas backend, rate limiting, navegacion por rol, proteccion de rutas frontend, manejo de token expirado, manejo de errores global frontend.

****Sprint 2**:** EPIC 03 (Categorias) + EPIC 04 (Ingredientes y Alergenos)

> CRUD completo de categorias (jerarquicas) e ingredientes con flag de alergenos.

****Sprint 3**:** EPIC 05 (Productos y Catalogo) + EPIC 06 (Perfil del Cliente)

> Alta, edicion, eliminacion de productos, asociacion a categorias e ingredientes, catalogo publico, filtros por alergenos. Perfil del cliente: ver, editar, cambiar contrasena.

****Sprint 4**:** EPIC 07 (Direcciones de Entrega) + EPIC 08 (Carrito de Compras)

> CRUD de direcciones, direccion predeterminada. Carrito client-side con Zustand: agregar, personalizar, modificar, eliminar, vaciar.

****Sprint 5**:** EPIC 09 (Validaciones Pre-Checkout) + EPIC 10 (Creacion de Pedidos)

> Validacion de disponibilidad y precios al checkout. Creacion atomica de pedidos con snapshots de precio y direccion, validacion de stock.

****Sprint 6****: EPIC 11 (Pagos con MercadoPago) + EPIC 12 (FSM de Pedidos)

> Integracion con MercadoPago Orders API, webhooks IPN, consulta de estado de pago, reintento. Maquina de estados completa: PENDIENTE -> CONFIRMADO -> EN_PREPARACION -> EN_CAMINO -> ENTREGADO, cancelacion, auditoria.

****Sprint 7****: EPIC 13 (Visualizacion de Pedidos) + EPIC 14 (Notificaciones y Feedback UX)

> Historial de pedidos del cliente, detalle de pedido, panel de gestion para Gestor de Pedidos. Confirmacion visual de pedido creado, feedback de retorno de MercadoPago.

****Sprint 8****: EPIC 15 (Admin Usuarios) + EPIC 16 (Catalogo Admin) + EPIC 17 (Metricas y Dashboard)

> Panel de usuarios (listar, editar, desactivar). Acceso Admin a catalogo y pedidos. Dashboard de metricas: ventas, ranking productos, distribucion por estado. Configuracion del sistema.